project needs is to ensure that you do not depend on any implicit library leaking from the surrounding environment.

NOTE Apart from dependency management, Gradle and Maven offer additional features for building, testing, and configuring a Java project, which are fundamental for application development. All the examples in the book will use Gradle, but feel free to use Maven instead.

Even though you have a dependency manifest in place, you'll still need to provide the dependency manager itself. Both Gradle and Maven offer a feature to run the tool from a *wrapper script* named `gradlew` or `mvnw` that you can include in your codebase. For example, rather than running a Gradle command like `gradle build` (which assumes you have Gradle installed on your machine), you can run `./gradlew build`. The script invokes the specific version of the build tool defined in the project. If the build tool is not present yet, the wrapper script will download it first and then run the command. Using the wrapper, you can ensure that all team members and automated tools building the project use the same Gradle or Maven version. When you're generating a new project from Spring Initializr, you'll also get a wrapper script that's ready to use, so you don't need to download or configure anything.

NOTE No matter what, you'll usually have at least one external dependency: the runtime. In our case, that's the Java Runtime Environment (JRE). If you package your application as a container image, the Java runtime will be included in the image itself, granting you more control over it. On the other hand, the final application artifact will depend on the container runtime required to run the image. You'll learn more about the containerization process in chapter 6.

Now, on to the code. The Polar Bookshop system has a Catalog Service application that's responsible for managing the books available in the catalog. In the previous chapter, we initialized the project. The architecture of the system is shown again in figure 3.2.